

Exqutor 기반 캡스톤 연구 방향 설계안

작성일: 2026-03-28 목적: 팀 토의 → 연구제안서(4/2 마감) 초안의 근거 자료 배경: Exqutor 논문의 기초를 유지하면서, 논문이 다루지 않은 영역을 확장 실험하는 방향

I. 출발점 — Exqutor가 해결한 것과 남겨놓은 것

Exqutor(arXiv:2512.09695v2)는 벡터 증강 분석 쿼리(VAQ)에서 카디널리티 오추정 문제를 해결했다. pgvector는 벡터 조건의 선택도를 33.3%로 고정 추정하고, VBASE는 50%, DuckDB는 100%로 추정하는데, 실제 선택도는 0.001%~90% 까지 다양하므로 옵티마이저가 잘못된 실행 계획을 선택하게 된다. Exqutor는 두 가지 해법을 제시했다.

- ECQO**: HNSW 인덱스가 있을 때, range query를 실행해 1~2ms 안에 정확한 카디널리티를 획득
- Adaptive Sampling**: 인덱스가 없을 때, 모멘텀 기반 동적 샘플링으로 카디널리티를 근사

그 결과 pgvector에서 최대 1,000배, VBASE에서 10,000배, DuckDB에서 1.5~37배 성능 향상을 달성했다.

논문의 실험 조건 (고정된 변수들)

변수	논문이 사용한 값	현실과의 괴리
쿼리 유형	range query (<code>WHERE distance < D</code>)	실무 RAG는 거의 전부 top-k
필터 조건	없음 또는 태그 1개	실무는 다중 조건 복합 필터
벡터-속성 관계	랜덤 부착 (독립)	현실은 유사 벡터=유사 속성
HNSW 파라미터	M=16, ef_c=200, ef_s=400 고정	실무는 메모리/성능 트레이드오프로 다양
벡터 차원	96~192d (DEEP, SIFT)	OpenAI 1536d, Cohere 4096d
거리 함수	L2(유클리드)만	코사인 유사도가 더 흔함
데이터 상태	정적	INSERT/UPDATE 지속 발생
pgvector 버전	0.7.x 시점	0.8.0에서 iterative scan 등 개선
필터 순서	옵티마이저에 위임	순서에 따라 성능 수배 차이

논문이 직접 인정한 한계 (Section 6.8)

- 고차원 공간에서 샘플링 오버헤드 증가 — 정량화하지 않음
- 기존 비용 모델의 부정확성 — 정확한 카디널리티를 줘도 비용 모델이 ANN 인덱스 특성을 반영 못함
- 벡터 인덱스 맞춤 비용 모델 부재 — 향후 과제로만 언급

II. 핵심 연구 질문

"Exqutor가 카디널리티를 정확하게 추정한 다음, 그 정보를 활용해 다중 조건 VAQ에서 필터 적용 순서를 최적화하면 추가적인 성능 향상이 가능한가?"

이 질문은 Exqutor의 성과를 부정하는 게 아니라, 그 성과를 **한 단계 더 활용**하는 방향이다. 논문은 "카디널리티를 정확히 추정하면 실행 계획이 좋아진다"까지 보여줬고, 우리는 "정확한 카디널리티로 **구체적으로 무엇을** 더 잘할 수 있는가"를 실험적으로 밝힌다.

하위 연구 질문

RQ1: 다중 조건 VAQ에서 필터 적용 순서가 실행 성능에 얼마나 영향을 미치는가? **RQ2:** Exqutor의 정확한 카디널리티 추정이 필터 순서 결정을 실제로 개선하는가? **RQ3:** 논문이 다루지 않은 조건(top-k, 복합 필터, 고차원 등)에서 Exqutor의 효과는 어떻게 변하는가?

III. 연구 방향 — 3단계 구조

1단계: Baseline 구축 — "현재 pgvector가 얼마나 틀리는가"

목적: 우리 실험 환경에서의 현황 파악. 이후 모든 실험의 비교 기준. Exqutor 패치 없이 순수 pgvector만으로 수행 가능하므로 환경 구축 초기 단계에서 바로 시작할 수 있다.

실험 내용: - 거리 임계값 D를 조절하여 벡터 선택도를 0.01% → 0.1% → 1% → 5% → 10% → 33% → 50% → 90%로 sweep - 각 선택도 지점에서 EXPLAIN ANALYZE 수집: - pgvector 추정 행 수 vs 실제 행 수 → Q-error 계산 - 선택된 실행 계획 (Index Scan / Seq Scan / 조인 방식) - 실제 실행 시간 (p50, p95)

핵심 산출물: - 선택도 vs Q-error 곡선 — "33.3% 고정 추정이 우연히 맞는 교차점" 시각화 - 선택도 vs 실행 계획 전환점 (crossover point) — "어디서 Index Scan↔Seq Scan이 바뀌어야 하는가"

논문과의 차별점: 논문은 TPC-H VAQ 벤치마크 결과만 보여주고, 선택도를 연속적으로 변화시키며 체계적으로 분석하지는 않았다. 특히 crossover point 매핑은 논문에 없는 분석이다.

참고 문헌 근거: - [6] Context-Enhanced Relational Operators: 선택도 0~10%에서는 스캔, 10~30%에서는 인덱스, 50%+에서는 다시 스캔이 유리하다는 전환점 존재를 확인 - [76] ACORN: 선택도에 따른 최적 전략(filter-first, search-first, hybrid)이 완전히 달라짐을 입증 - [77] SERF: 선택도 오추정 ±30%에서 1.3배, ±50%에서 2배 성능 저하 관측

2단계: 핵심 실험 — 필터 순서 최적화 (재현 아이디어의 학술적 검증)

배경: 재현(강재현)의 아이디어 — "많이 retrieval되는 조건을 먼저 적용하고, 적게 retrieval되는 조건을 나중에 적용하면 어떨까?" 이것을 학술적으로 정식화하면 **다중 조건 VAQ에서의 Filter Ordering 최적화** 문제가 된다.

학술적 맥락: 관계형 DB의 조인 순서 최적화(Join Ordering)는 수십 년 연구된 고전 문제다. 그러나 벡터 조건이 포함된 복합 쿼리에서의 필터 순서 최적화는 거의 연구되지 않았다. 그 이유는 벡터 조건의 선택도를 정확히 알 수 없었기 때문이다. Exqutor가 이 문제를 해결한 지금, 필터 순서 최적화가 비로소 가능해졌다.

실험 설계:

(a) 쿼리 구조: Exqutor의 TPC-H VAQ 벤치마크(Q3, Q5, Q8 등)를 기반으로 관계형 필터를 단계적으로 추가한다. 아래는 필터 단계 구조의 예시이다.

```
-- Level 0: 벡터 조건만 (Exqutor 원본 VAQ)
SELECT ... WHERE embedding <-> :query < :D

-- Level 1: + 관계형 필터 1개
SELECT ... WHERE embedding <-> :query < :D
      AND l_shipdate > '1995-03-15'

-- Level 2: + 관계형 필터 2개
SELECT ... WHERE embedding <-> :query < :D
      AND l_shipdate > '1995-03-15'
      AND o_orderpriority = '1-URGENT'

-- Level 3: + 관계형 필터 3개 + 조인
SELECT ... FROM lineitem l JOIN orders o ON l.l_orderkey = o.o_orderkey
WHERE l.embedding <-> :query < :D
      AND l.l_shipdate > '1995-03-15'
      AND o.o_orderpriority = '1-URGENT'
      AND o.o_totalprice > 50000
```

(b) 필터 순서 변형: pg_hint_plan으로 강제 지정

전략	설명	재현 아이디어와의 관계
V→R	벡터 조건 먼저, 관계형 나중	—
R→V	관계형 조건 먼저, 벡터 나중	—
선택도 ↑→↓	선택도 높은(많이 남기는) 것 먼저	재현의 "많이→적게"
선택도 ↓→↑	선택도 낮은(적게 남기는) 것 먼저	재현의 역순
옵티마이저 기본	pgvector 기본 판단	baseline
ECQO + 기본	정확한 카디널리티 제공 후 옵티마이저 판단	Exqutor 적용

(c) 실험 매트릭스: 필터 순서 6가지 × 벡터 선택도 4단계(0.1%, 1%, 10%, 50%) × 필터 개수 4단계(0~3개) = 최대 96개 조합

각 조합에서 측정: - 실행 시간 (5회 반복, 중앙값) - 실행 계획 구조 (EXPLAIN JSON 파싱) - 각 필터 단계별 중간 카디널리티

핵심 산출물: - "벡터 선택도 × 필터 순서" 최적 전략 히트맵: 어떤 선택도에서 어떤 순서가 최적인지 - "필터 개수 vs 순서 효과" 곡선: 필터가 많아질수록 순서 최적화 효과가 커지는지 - ECQO 적용 전후 비교: 정확한 카디널리티가 순서 결정을 실제로 개선하는지

논문과의 차별점: Exqutor 논문은 카디널리티 추정의 정확도를 높이는 것 자체에 집중했다. 필터 순서 최적화는 다루지 않았으며, 다중 필터 조합 실험도 단일 태그 필터에 한정되었다. 우리 실험은 "정확한 카디널리티가 옵티마이저의 필터 순서 결정을 어떻게 개선하는가"라는 다음 단계의 질문에 답한다.

참고 문헌 근거: - [41] Filtered-DiskANN: 선택도별 검색 범위를 $K \times 2 (>50\%)$, $K \times 5 (>10\%)$, $K \times 20 (>1\%)$, $K \times 100 (<1\%)$ 로 동적 조정 — 선택도에 따른 전략 분기의 중요성 - [74] Consistent Flexible Selectivity: 복합 필터 선택도의 "selectivity(A AND B) = selectivity(A) × P(B|A)" 공리 — 독립성 가정이 깨질 때의 오차 분석 프레임워크 - [54] Datta et al.: 카디널리티 추정 오류 10배 → 실행시간 100배 증가 관측

3단계: 확장 실험 — 논문이 고정한 변수를 하나씩 바꾸기

1단계와 2단계가 핵심이고, 3단계는 여력에 따라 1~2개를 선택하여 추가한다. 모든 확장 실험은 2단계의 필터 순서 실험 위에 변수를 하나 추가하는 형태로, 독립적으로 수행 가능하다.

3-A. top-k로의 전환

질문: range query에서 확인한 필터 순서 효과가 top-k(`ORDER BY distance LIMIT k`)에서도 유효한가?

방법: 2단계와 동일한 쿼리를 top-k 버전으로 변환하여 반복 실행. $k=10, 100, 1000$ 에서 각각 필터 순서별 성능 비교.

기대: top-k에서는 k 가 카디널리티를 제한하므로 필터 순서 효과가 약해질 수 있지만, 조인이 포함된 복잡한 쿼리에서는 여전히 유의미할 것으로 예상.

논문과의 차별점: Exqutor가 의도적으로 제외한 영역. "top-k에서는 카디널리티 문제가 다른 형태로 나타나는가?"에 대한 실험적 답변.

3-B. HNSW 파라미터 변화

질문: HNSW의 M 과 ef_search 가 낮아지면 ECQO의 range query 결과 자체가 부정확해지는가?

방법: $M=4/8/16/32$, $ef_search=40/100/200/400$ 에서 (1) ECQO 카디널리티 정확도, (2) 2단계 필터 순서 실험 결과 변화를 동시 측정.

기대: ef_search 가 낮으면 HNSW가 모든 이웃을 찾지 못해 ECQO 카디널리티가 실제보다 낮게 나올 수 있다. 이 부정확성이 필터 순서 결정에 미치는 영향을 정량화.

논문과의 차별점: 논문은 $M=16$, $ef_s=400$ 고정. 실무에서 메모리 절약을 위해 파라미터를 낮추는 상황에서의 ECQO 신뢰도는 미검증.

참고 문헌 근거: [72] Vector Search Performance Index Size — 인덱스 파라미터 축소 시 recall 하락과 성능 트레이드오프 분석 프레임워크 차용 가능.

3-C. 고차원 벡터 (1536d+)

질문: 논문이 인정한 한계 #1 — 고차원에서 ECQO 오버헤드가 얼마나 증가하는가?

방법: sentence-transformers(768d) 또는 OpenAI ada-002(1536d) 임베딩으로 동일 실험 반복. ECQO의 오버헤드(ms)와 카디널리티 정확도를 차원별로 측정.

기대: 차원이 높아질수록 HNSW의 거리 계산 비용이 증가하여 ECQO 오버헤드가 선형 이상으로 증가할 가능성. "1~2ms"가 유지되는 차원의 상한선을 특정.

논문과의 차별점: 논문 데이터셋은 96~192d. 1536d 이상에서의 ECQO 오버헤드 정량화는 미수행.

3-D. pgvector 0.8.0 비교

질문: pgvector 0.8.0의 자체 개선(iterative scan, 비용 추정 공식 수정)으로 Exqutor의 필요성이 줄어들었는가?

방법: 동일한 1~2단계 실험을 pgvector 0.8.0에서 반복. 특히 iterative scan이 활성화된 상태에서의 필터링 성능과 ECQO의 효과를 비교.

기대: iterative scan이 post-filter 문제를 부분적으로 해결하지만, 카디널리티 추정 자체는 여전히 부정확할 것으로 예상. ECQO와 iterative scan이 상호보완적인지 대체적인지 판별.

논문과의 차별점: 논문 시점(pgvector 0.7.x) 이후 pgvector 자체가 진화했다. 이 진화가 Exqutor의 가치를 어떻게 변화시키는지의 시의성 있는 질문.

참고: pgvector GitHub PR #682, #686에서 비용 추정 공식 수정 내역 확인 가능. Issue #835에서 0.8.0에서도 HNSW 인덱스 미사용 문제가 보고됨.

3-E. 거리 함수 변경 (L2 → 코사인 → 내적)

질문: 거리 함수가 바뀌면 카디널리티 추정 패턴과 필터 순서 효과가 달라지는가?

방법: 동일 데이터에서 L2(<->), 코사인(<=>), 내적(<#>) 각각으로 인덱스를 빌드하고 1~2단계 실험 반복.

기대: 정규화된 벡터에서 L2와 코사인은 동치이므로 차이가 크지 않을 수 있지만, 내적(inner product)은 범위의 의미가 달라지므로 카디널리티 분포가 변할 가능성.

3-F. 벡터-속성 상관관계

질문: 벡터와 관계형 속성 사이에 상관관계가 있으면 카디널리티 추정 오차가 커지는가, 작아지는가?

방법: TPC-H 데이터에 벡터를 부착할 때, (a) 랜덤 부착 (논문 방식), (b) 카테고리별 클러스터 벡터 생성 (유사 상품=유사 벡터), (c) 가격 비례 벡터 생성 — 세 가지로 2단계 실험 반복.

기대: 상관관계가 있으면 옵티마이저의 독립성 가정($\text{selectivity}(A \text{ AND } B) = \text{selectivity}(A) \times \text{selectivity}(B)$)이 깨져서 복합 필터 카디널리티 추정 오차가 증폭될 가능성.

IV. 실험 환경

소프트웨어

구성 요소	버전/설정
PostgreSQL	16 (Exqutor 패치 적용 빌드)
pgvector	0.7.x (논문 재현용) + 0.8.0 (비교용)
pg_hint_plan	최신 (필터 순서 강제 지정용)
Python	3.10+ (벤치마크 스크립트, 결과 분석)
TPC-H	SF1 또는 SF10 (데이터 규모는 하드웨어에 따라 결정)

하드웨어 (미확정)

- 옵션 A: 연구실 서버 (교수님께 요청 필요)
- 옵션 B: 팀원 개인 머신 (소규모 SF1로 시작)
- 결정 필요: 세인이 교수님께 서버 사용 가능 여부 확인 중

데이터셋

- 기본: TPC-H VAQ 벤치마크 (Exqutor GitHub의 `Vector-augmented_SQL_analytics/` 스크립트 활용)
- 벡터 데이터: SIFT1M(128d) 기본, sentence-transformers(768d) 또는 OpenAI(1536d) 확장용
- 데이터 로딩: Exqutor GitHub의 `insert_data_pgvector.sh` 활용

Exqutor 코드 상태 (GitHub 확인 결과)

- 구조: PostgreSQL 16 소스에 패치 파일 적용 → 수정된 PostgreSQL 빌드
- 빌드 방법: `apply_patch.sh` → `build.sh`
- 벤치마크: TPC-H VAQ 8개 쿼리(Q3, Q5, Q8, Q9, Q10, Q11, Q12, Q20) + 데이터셋 다운로드 스크립트
- 마지막 커밋: 2025-11-12 (paper artifact 수준, 활발한 유지보수 없음)

V. 예상 일정

기간	마일스톤	산출물
~4/2	연구제안서 제출	연구제안서 PDF (4페이지)
4/1~4/14	환경 구축	PostgreSQL+pgvector+pg_hint_plan 설치, TPC-H 데이터 로딩, EXPLAIN 파싱 스크립트
4/14~4/28	1단계 실험 (Baseline)	선택도 sweep 결과, Q-error 곡선, crossover point 매핑
4/28~5/15	2단계 실험 (필터 순서)	필터 순서별 성능 비교, 최적 전략 히트맵
5/15~5/25	3단계 확장 (택 1~2개)	확장 실험 결과
5월 말	중간발표 + 중간보고서	발표 자료, 보고서
6월	결과 정리 + 최종 보고서	최종발표, 최종보고서, 전시회

VI. 예상 산출물 요약

- 선택도 vs Q-error 곡선 + crossover point 맵 — 1단계
- "벡터 선택도 × 필터 순서" 최적 전략 히트맵 — 2단계 핵심
- "필터 개수 vs 순서 최적화 효과" 곡선 — 2단계
- ECQO 적용 전후 필터 순서 결정 품질 비교 — 2단계
- 확장 실험 결과 (top-k, 파라미터, 고차원 등) — 3단계
- 실무 가이드라인: "이런 조건에서는 이 순서로 필터를 적용하라" — 종합

VII. 스토리라인 (연구제안서 서사 구조)

벡터 증강 분석 쿼리(VAQ)는 벡터 유사도 검색과 관계형 분석을 결합하는 차세대 데이터 분석의 핵심이다. 그러나 기존 RDBMS의 쿼리 옵티마이저는 벡터 조건의 선택도를 정확히 추정하지 못하여 잘못된 실행 계획을 생성하는 문제가 있으며, Exqutor(2024)가 ECQO와 Adaptive Sampling을 통해 이 카디널리티 추정 문제를 해결했다.

그러나 실제 VAQ는 벡터 조건 하나만으로 구성되지 않는다. 날짜 범위, 카테고리, 가격 등 다수의 관계형 필터가 벡터 조건과 복합적으로 결합되며, 이때 **어떤 필터를 어떤 순서로 적용하느냐**가 실행 성능을 크게 좌우한다. 이 필터 순서 결정은 각 필터의 선택도를 정확히 알아야만 올바르게 내릴 수 있으므로, Exqutor의 정확한 카디널리티 추정은 필터 순서 최적화의 전제 조건이 된다.

본 연구는 Exqutor의 기초 위에서, 다중 조건 VAQ에서의 필터 순서 최적화 효과를 체계적으로 분석한다. 구체적으로, (1) pgvector에서 선택도별 실행 계획 전환점을 매핑하고, (2) 다양한 필터 순서 전략의 성능을 정량 비교하며, (3) Exqutor의 ECQO가 이 순서 결정을 실제로 개선하는지 검증한다. 나아가 top-k 쿼리, 고차원 벡터, pgvector 최신 버전 등 논문이 다루지 않은 조건에서의 확장 실험을 수행한다.

VIII. 팀 작업 분담

4인 1팀으로 운영하되, 단계별로 역할을 유동적으로 배분한다.

환경 구축 (4/1~4/14, 전원): - PostgreSQL 16 + Exqutor 패치 빌드 (소스 컴파일) - pgvector, pg_hint_plan 설치 - TPC-H VAQ 데이터 생성 및 로딩 - EXPLAIN ANALYZE 결과 자동 수집 Python 스크립트 구축 - 벡터 데이터셋(SIFT1M) 로딩 및 인덱스 빌드

1~2단계 실험 (4/14~5/15, 전원): - 쿼리 설계 및 선택도 sweep 파라미터 세팅 - 실험 실행 및 결과 수집 - EXPLAIN JSON 파싱 및 시각화 - 결과 분석 및 해석

3단계 확장 + 보고서 (5/15~6월, 전원): - 확장 실험 택 1~2개 수행 - 중간발표 자료 작성 - 최종보고서 작성

IX. 토의 포인트 (팀원 확인 필요)

- [최우선] pgvector에서 range query가 HNSW 인덱스를 타는지 확인** — `WHERE embedding <-> query < D` 형태가 인덱스를 쓰는지. pgvector는 기본적으로 top-k(`ORDER BY ... LIMIT k`) 중심이고, range query는 Seq Scan으로 빠질 가능성이 있다. 이게 확인 안 되면 1~2단계 실험 설계 전체를 top-k 기반으로 조정해야 한다.
- Exqutor 코드 빌드 가능 여부** — 누가 먼저 clone해서 `apply_patch.sh && build.sh` 돌려볼 것인가
- 연구실 서버 사용 가능 여부** — 세은이 교수님께 확인 중. 결과에 따라 데이터 규모(SF1 vs SF10) 결정
- 3단계 확장 실험 우선순위** — top-k(3-A)가 가장 실무 관련성 높고, pgvector 0.8.0 비교(3-D)가 시의성 높음. 둘 중 하나 또는 둘 다?
- 연구제안서 4페이지 구성** — 위 스토리라인으로 가되, 3단계 확장은 "향후 계획"으로 넣을지, 핵심 실험으로 넣을지